

> **LAB 1**

↳ 6 cells hidden

> **LAB 2**

↳ 11 cells hidden

> **LAB 3**

↳ 4 cells hidden

> **LAB 4**

↳ 20 cells hidden

> **LAB 5**

↳ 1 cell hidden

> **LAB 6**

▶ ↳ 2 cells hidden

✓ **LAB 7**

✓ **House price prediction**

```
# =====  
# 1. Import Libraries  
# =====
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1])),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

```

```

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")

```

```
plt.ylabel("Predicted Prices")  
plt.title("Actual vs Predicted")  
plt.show()
```



```
Dataset Shape: (20640, 9)
  MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252    41.0  6.984127  1.023810    322.0  2.555556    37.88
1  8.3014    21.0  6.238137  0.971880   2401.0  2.109842    37.86
2  7.2574    52.0  8.288136  1.073446    496.0  2.802260    37.85
3  5.6431    52.0  5.817352  1.073059    558.0  2.547945    37.85
4  3.8462    52.0  6.281853  1.081081    565.0  2.181467    37.85

  Longitude  Price
0   -122.23  4.526
1   -122.22  3.585
2   -122.24  3.521
3   -122.25  3.413
4   -122.25  3.422
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Use
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,152
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

```
Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/100
465/465 ██████████ 6s 6ms/step - loss: 1.1009 - mae: 0.7195 - val_lo
Epoch 2/100
465/465 ██████████ 2s 4ms/step - loss: 0.3773 - mae: 0.4342 - val_lo
Epoch 3/100
465/465 ██████████ 1s 2ms/step - loss: 0.3555 - mae: 0.4214 - val_lo
Epoch 4/100
465/465 ██████████ 1s 2ms/step - loss: 0.3210 - mae: 0.3943 - val_lo
Epoch 5/100
465/465 ██████████ 2s 3ms/step - loss: 0.3145 - mae: 0.3918 - val_lo
Epoch 6/100
465/465 ██████████ 2s 3ms/step - loss: 0.3116 - mae: 0.3891 - val_lo
Epoch 7/100
465/465 ██████████ 1s 2ms/step - loss: 0.3078 - mae: 0.3840 - val_lo
Epoch 8/100
465/465 ██████████ 1s 2ms/step - loss: 0.2955 - mae: 0.3795 - val_lo
Epoch 9/100
465/465 ██████████ 1s 2ms/step - loss: 0.3086 - mae: 0.3843 - val_lo
Epoch 10/100
465/465 ██████████ 1s 2ms/step - loss: 0.3000 - mae: 0.3819 - val_lo
Epoch 11/100
465/465 ██████████ 1s 2ms/step - loss: 0.2939 - mae: 0.3761 - val_lo
Epoch 12/100
465/465 ██████████ 1s 2ms/step - loss: 0.2763 - mae: 0.3621 - val_lo
Epoch 13/100
465/465 ██████████ 1s 2ms/step - loss: 0.2842 - mae: 0.3685 - val_lo
```

```

465/465 ————— 1s 2ms/step - loss: 0.2842 - mae: 0.3685 - val_lo
Epoch 14/100
465/465 ————— 1s 2ms/step - loss: 0.2671 - mae: 0.3577 - val_lo
Epoch 15/100
465/465 ————— 1s 2ms/step - loss: 0.2780 - mae: 0.3627 - val_lo
Epoch 16/100
465/465 ————— 1s 2ms/step - loss: 0.2697 - mae: 0.3537 - val_lo
Epoch 17/100
465/465 ————— 2s 3ms/step - loss: 0.2658 - mae: 0.3529 - val_lo
Epoch 18/100
465/465 ————— 2s 3ms/step - loss: 0.2576 - mae: 0.3487 - val_lo
Epoch 19/100
465/465 ————— 1s 3ms/step - loss: 0.2520 - mae: 0.3454 - val_lo
Epoch 20/100
465/465 ————— 1s 2ms/step - loss: 0.2681 - mae: 0.3559 - val_lo
Epoch 21/100
465/465 ————— 1s 2ms/step - loss: 0.2495 - mae: 0.3448 - val_lo
Epoch 22/100
465/465 ————— 1s 2ms/step - loss: 0.2570 - mae: 0.3469 - val_lo
Epoch 23/100
465/465 ————— 1s 2ms/step - loss: 0.2638 - mae: 0.3474 - val_lo
Epoch 24/100
465/465 ————— 1s 2ms/step - loss: 0.2580 - mae: 0.3481 - val_lo
Epoch 25/100
465/465 ————— 1s 2ms/step - loss: 0.2553 - mae: 0.3428 - val_lo
Epoch 26/100
465/465 ————— 1s 2ms/step - loss: 0.2445 - mae: 0.3363 - val_lo
Epoch 27/100
465/465 ————— 1s 2ms/step - loss: 0.2324 - mae: 0.3322 - val_lo
Epoch 28/100
465/465 ————— 1s 2ms/step - loss: 0.2536 - mae: 0.3443 - val_lo
Epoch 29/100
465/465 ————— 1s 3ms/step - loss: 0.2471 - mae: 0.3386 - val_lo
Epoch 30/100
465/465 ————— 3s 3ms/step - loss: 0.2358 - mae: 0.3330 - val_lo
Epoch 31/100
465/465 ————— 1s 2ms/step - loss: 0.2449 - mae: 0.3322 - val_lo
Epoch 32/100
465/465 ————— 1s 2ms/step - loss: 0.2286 - mae: 0.3269 - val_lo
Epoch 33/100
465/465 ————— 1s 2ms/step - loss: 0.2389 - mae: 0.3361 - val_lo
Epoch 34/100
465/465 ————— 1s 2ms/step - loss: 0.2319 - mae: 0.3287 - val_lo
Epoch 35/100
465/465 ————— 1s 2ms/step - loss: 0.2246 - mae: 0.3230 - val_lo
Epoch 36/100
465/465 ————— 1s 2ms/step - loss: 0.2232 - mae: 0.3215 - val_lo
Epoch 37/100
465/465 ————— 1s 2ms/step - loss: 0.2318 - mae: 0.3294 - val_lo
Epoch 38/100
465/465 ————— 1s 2ms/step - loss: 0.2408 - mae: 0.3326 - val_lo
Epoch 39/100
465/465 ————— 1s 2ms/step - loss: 0.2303 - mae: 0.3285 - val_lo
Epoch 40/100
465/465 ————— 1s 2ms/step - loss: 0.2191 - mae: 0.3193 - val_lo
Epoch 41/100
465/465 ————— 2s 3ms/step - loss: 0.2211 - mae: 0.3253 - val_lo

```

```

Epoch 42/100
465/465  2s 4ms/step - loss: 0.2247 - mae: 0.3211 - val_lo
Epoch 43/100
465/465  2s 3ms/step - loss: 0.2178 - mae: 0.3161 - val_lo
Epoch 44/100
465/465  1s 2ms/step - loss: 0.2212 - mae: 0.3219 - val_lo
Epoch 45/100
465/465  1s 2ms/step - loss: 0.2174 - mae: 0.3185 - val_lo
Epoch 46/100
465/465  1s 2ms/step - loss: 0.2077 - mae: 0.3132 - val_lo
Epoch 47/100
465/465  1s 2ms/step - loss: 0.2112 - mae: 0.3156 - val_lo
Epoch 48/100
465/465  1s 2ms/step - loss: 0.2132 - mae: 0.3151 - val_lo
Epoch 49/100
465/465  1s 2ms/step - loss: 0.2061 - mae: 0.3086 - val_lo
Epoch 50/100
465/465  1s 2ms/step - loss: 0.2143 - mae: 0.3154 - val_lo
Epoch 51/100
465/465  1s 2ms/step - loss: 0.2111 - mae: 0.3151 - val_lo
Epoch 52/100
465/465  1s 2ms/step - loss: 0.1989 - mae: 0.3063 - val_lo
Epoch 53/100
465/465  1s 2ms/step - loss: 0.2049 - mae: 0.3124 - val_lo
Epoch 54/100
465/465  1s 3ms/step - loss: 0.1999 - mae: 0.3057 - val_lo
Epoch 55/100
465/465  3s 3ms/step - loss: 0.2106 - mae: 0.3148 - val_lo
Epoch 56/100
465/465  1s 2ms/step - loss: 0.2029 - mae: 0.3078 - val_lo
Epoch 57/100
465/465  1s 2ms/step - loss: 0.2003 - mae: 0.3040 - val_lo
Epoch 58/100
465/465  1s 2ms/step - loss: 0.1986 - mae: 0.3061 - val_lo
Epoch 59/100
465/465  1s 2ms/step - loss: 0.1971 - mae: 0.3057 - val_lo
Epoch 60/100
465/465  1s 2ms/step - loss: 0.2015 - mae: 0.3111 - val_lo
Epoch 61/100
465/465  1s 2ms/step - loss: 0.1959 - mae: 0.3066 - val_lo
Epoch 62/100
465/465  1s 2ms/step - loss: 0.1945 - mae: 0.3034 - val_lo
Epoch 63/100
465/465  1s 2ms/step - loss: 0.1841 - mae: 0.2988 - val_lo
Epoch 64/100
465/465  1s 2ms/step - loss: 0.1958 - mae: 0.3048 - val_lo
Epoch 65/100
465/465  1s 2ms/step - loss: 0.2017 - mae: 0.3071 - val_lo
Epoch 66/100
465/465  2s 4ms/step - loss: 0.1915 - mae: 0.3000 - val_lo
Epoch 67/100
465/465  2s 3ms/step - loss: 0.1892 - mae: 0.2991 - val_lo
Epoch 68/100
465/465  1s 3ms/step - loss: 0.1839 - mae: 0.2923 - val_lo
Epoch 69/100
465/465  1s 2ms/step - loss: 0.1981 - mae: 0.3007 - val_lo
Epoch 70/100
465/465  1s 2ms/step - loss: 0.1981 - mae: 0.3007 - val_lo

```



```

Epoch 70/100
465/465 ██████████ 1s 2ms/step - loss: 0.1933 - mae: 0.3042 - val_lo
Epoch 71/100
465/465 ██████████ 2s 3ms/step - loss: 0.1777 - mae: 0.2902 - val_lo
Epoch 72/100
465/465 ██████████ 1s 2ms/step - loss: 0.1870 - mae: 0.2972 - val_lo
Epoch 73/100
465/465 ██████████ 1s 2ms/step - loss: 0.1902 - mae: 0.3007 - val_lo
Epoch 74/100
465/465 ██████████ 1s 2ms/step - loss: 0.1790 - mae: 0.2930 - val_lo
Epoch 75/100
465/465 ██████████ 1s 2ms/step - loss: 0.1876 - mae: 0.2982 - val_lo
Epoch 76/100
465/465 ██████████ 1s 2ms/step - loss: 0.1788 - mae: 0.2894 - val_lo
Epoch 77/100
465/465 ██████████ 1s 2ms/step - loss: 0.1818 - mae: 0.2948 - val_lo
Epoch 78/100
465/465 ██████████ 2s 3ms/step - loss: 0.1773 - mae: 0.2904 - val_lo
Epoch 79/100
465/465 ██████████ 2s 3ms/step - loss: 0.1748 - mae: 0.2906 - val_lo
Epoch 80/100
465/465 ██████████ 2s 3ms/step - loss: 0.1744 - mae: 0.2920 - val_lo
Epoch 81/100
465/465 ██████████ 1s 2ms/step - loss: 0.1687 - mae: 0.2855 - val_lo
Epoch 82/100
465/465 ██████████ 1s 2ms/step - loss: 0.1794 - mae: 0.2933 - val_lo
Epoch 83/100
465/465 ██████████ 1s 2ms/step - loss: 0.1728 - mae: 0.2866 - val_lo
Epoch 84/100
465/465 ██████████ 1s 2ms/step - loss: 0.1756 - mae: 0.2906 - val_lo
Epoch 85/100
465/465 ██████████ 1s 2ms/step - loss: 0.1765 - mae: 0.2915 - val_lo
Epoch 86/100
465/465 ██████████ 1s 2ms/step - loss: 0.1753 - mae: 0.2906 - val_lo
Epoch 87/100
465/465 ██████████ 1s 2ms/step - loss: 0.1718 - mae: 0.2874 - val_lo
Epoch 88/100
465/465 ██████████ 1s 2ms/step - loss: 0.1598 - mae: 0.2797 - val_lo
Epoch 89/100
465/465 ██████████ 1s 2ms/step - loss: 0.1717 - mae: 0.2881 - val_lo
Epoch 90/100
465/465 ██████████ 1s 3ms/step - loss: 0.1615 - mae: 0.2779 - val_lo
Epoch 91/100
465/465 ██████████ 3s 4ms/step - loss: 0.1758 - mae: 0.2922 - val_lo
Epoch 92/100
465/465 ██████████ 1s 2ms/step - loss: 0.1717 - mae: 0.2853 - val_lo
Epoch 93/100
465/465 ██████████ 1s 2ms/step - loss: 0.1728 - mae: 0.2880 - val_lo
Epoch 94/100
465/465 ██████████ 1s 2ms/step - loss: 0.1632 - mae: 0.2825 - val_lo
Epoch 95/100
465/465 ██████████ 1s 2ms/step - loss: 0.1680 - mae: 0.2856 - val_lo
Epoch 96/100
465/465 ██████████ 1s 2ms/step - loss: 0.1632 - mae: 0.2832 - val_lo
Epoch 97/100
465/465 ██████████ 1s 2ms/step - loss: 0.1598 - mae: 0.2784 - val_lo
Epoch 98/100

```

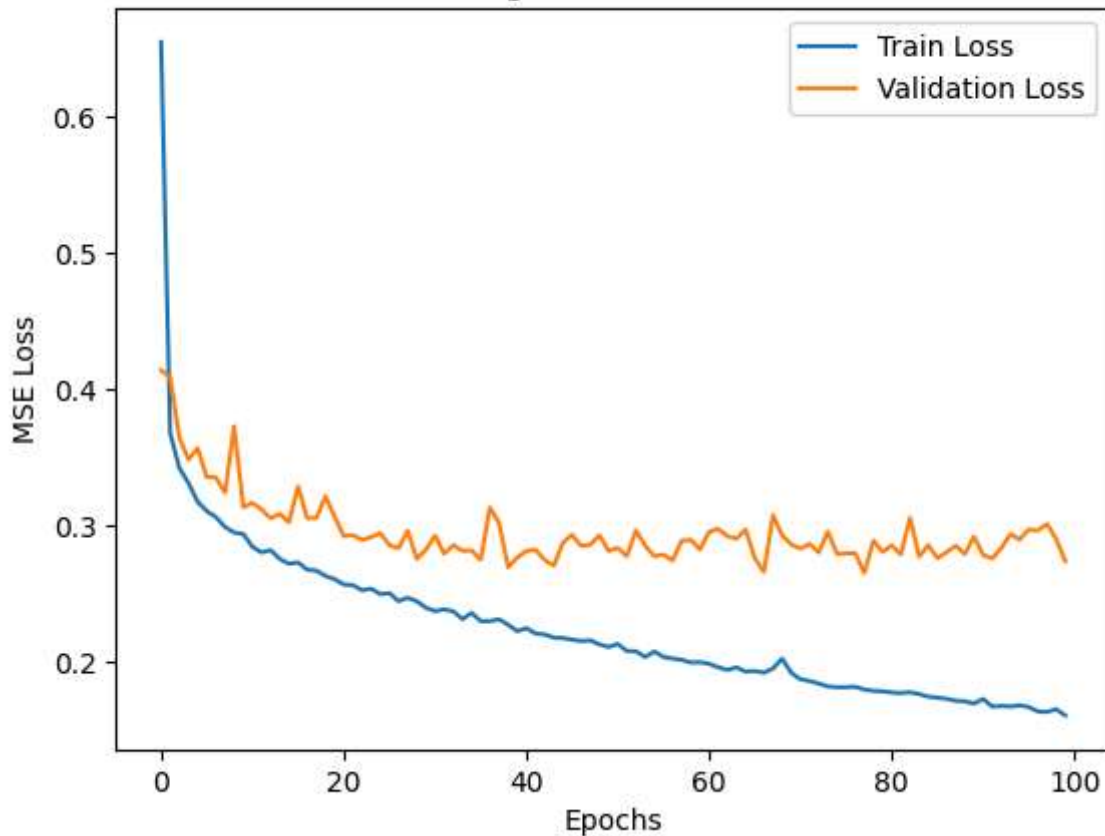
```
465/465 ————— 1s 2ms/step - loss: 0.1649 - mae: 0.2812 - val_lo  
Epoch 99/100  
465/465 ————— 1s 2ms/step - loss: 0.1617 - mae: 0.2795 - val_lo  
Epoch 100/100  
465/465 ————— 1s 2ms/step - loss: 0.1585 - mae: 0.2768 - val_lo  
129/129 ————— 0s 2ms/step - loss: 0.2680 - mae: 0.3408  
129/129 ————— 0s 1ms/step
```

Test MAE: 0.33989569544792175

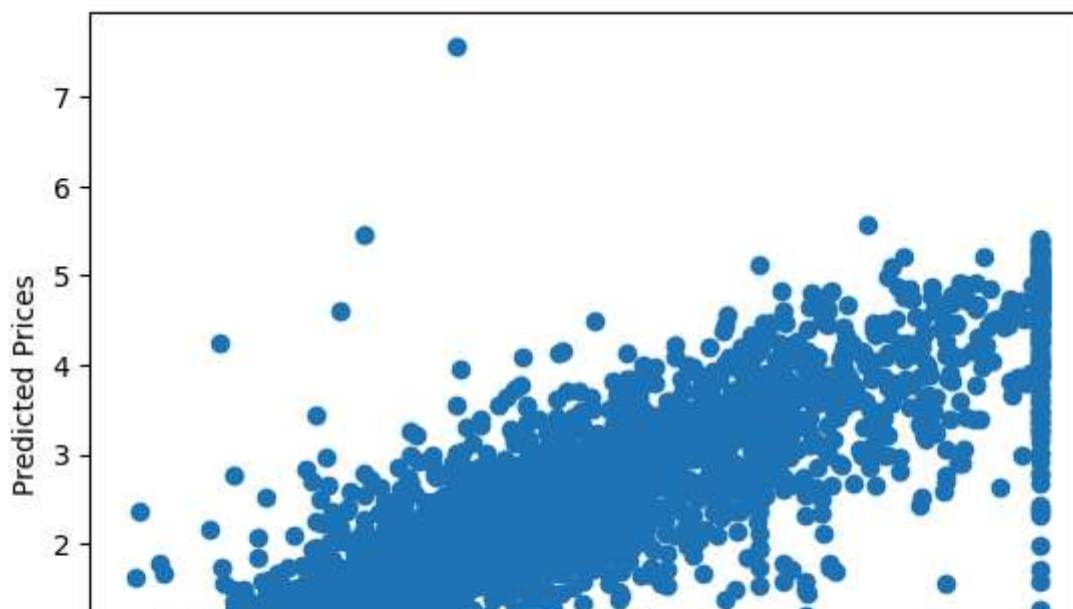
RMSE: 0.5190575038669664

R2 Score: 0.7943995372480752

Training vs Validation Loss



Actual vs Predicted



```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)

print("\nDataset Columns:")
print(df.columns.tolist())

print("\nFirst 5 Rows:")
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# =====
# 5. Feature Scaling
# =====
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("\nStandardScaler Parameters:")
```

```
scaler_df = pd.DataFrame({
    "Feature": X.columns,
    "Mean": scaler.mean_,
    "Std Dev": scaler.scale_
})

print(scaler_df)

# =====
# 6. Build Multilayer Neural Network
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1]
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)    # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.02),
    loss='mse',
    metrics=['mae']
)

print("\nModel Summary:")
model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train,
    y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=64,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
```

```
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training vs Validation Loss
# =====
plt.figure()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")

plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Housing Prices")

plt.show()
```


Dataset Shape: (20640, 9)

Dataset Columns:

['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Lat

First 5 Rows:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

StandardScaler Parameters:

	Feature	Mean	Std Dev
0	MedInc	3.876892	1.904842
1	HouseAge	28.575374	12.613197
2	AveRooms	5.438125	2.453484
3	AveBedrms	1.098033	0.447482
4	Population	1427.927326	1140.185729
5	AveOccup	3.119236	12.373207
6	Latitude	35.650669	2.135668
7	Longitude	-119.584102	2.002861

Model Summary:

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Use
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 128)	1,152
dense_17 (Dense)	(None, 64)	8,256
dense_18 (Dense)	(None, 32)	2,080
dense_19 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

204/204 ————— 4s 9ms/step - loss: 1.4049 - mae: 0.7896 - val_lo

Epoch 2/100

204/204 ————— 1s 3ms/step - loss: 0.3891 - mae: 0.4443 - val_lo

Epoch 3/100

204/204 ————— 1s 2ms/step - loss: 0.3632 - mae: 0.4260 - val_lo

Epoch 4/100

204/204 ————— 1s 2ms/step - loss: 0.3367 - mae: 0.4030 - val_lo

```

204/204 ————— 1s 3ms/step - loss: 0.3267 - mae: 0.4030 - val_lo
Epoch 5/100
204/204 ————— 1s 2ms/step - loss: 0.3649 - mae: 0.4162 - val_lo
Epoch 6/100
204/204 ————— 1s 3ms/step - loss: 0.3341 - mae: 0.4039 - val_lo
Epoch 7/100
204/204 ————— 1s 2ms/step - loss: 0.3413 - mae: 0.4077 - val_lo
Epoch 8/100
204/204 ————— 1s 4ms/step - loss: 0.3119 - mae: 0.3860 - val_lo
Epoch 9/100
204/204 ————— 1s 5ms/step - loss: 0.3137 - mae: 0.3875 - val_lo
Epoch 10/100
204/204 ————— 1s 5ms/step - loss: 0.3233 - mae: 0.3901 - val_lo
Epoch 11/100
204/204 ————— 1s 4ms/step - loss: 0.3060 - mae: 0.3881 - val_lo
Epoch 12/100
204/204 ————— 1s 5ms/step - loss: 0.3063 - mae: 0.3829 - val_lo
Epoch 13/100
204/204 ————— 1s 3ms/step - loss: 0.2999 - mae: 0.3727 - val_lo
Epoch 14/100
204/204 ————— 1s 3ms/step - loss: 0.3108 - mae: 0.3852 - val_lo
Epoch 15/100
204/204 ————— 1s 3ms/step - loss: 0.3242 - mae: 0.3923 - val_lo
Epoch 16/100
204/204 ————— 1s 3ms/step - loss: 0.3197 - mae: 0.3918 - val_lo
Epoch 17/100
204/204 ————— 1s 3ms/step - loss: 0.2992 - mae: 0.3767 - val_lo
Epoch 18/100
204/204 ————— 1s 3ms/step - loss: 0.3141 - mae: 0.3844 - val_lo
Epoch 19/100
204/204 ————— 1s 3ms/step - loss: 0.3124 - mae: 0.3887 - val_lo
Epoch 20/100
204/204 ————— 1s 2ms/step - loss: 0.2912 - mae: 0.3747 - val_lo
Epoch 21/100
204/204 ————— 1s 3ms/step - loss: 0.2769 - mae: 0.3612 - val_lo
Epoch 22/100
204/204 ————— 1s 3ms/step - loss: 0.2921 - mae: 0.3706 - val_lo
Epoch 23/100
204/204 ————— 1s 3ms/step - loss: 0.3135 - mae: 0.3835 - val_lo
Epoch 24/100
204/204 ————— 1s 3ms/step - loss: 0.2766 - mae: 0.3640 - val_lo
Epoch 25/100
204/204 ————— 1s 3ms/step - loss: 0.2851 - mae: 0.3686 - val_lo
Epoch 26/100
204/204 ————— 1s 3ms/step - loss: 0.2779 - mae: 0.3624 - val_lo
Epoch 27/100
204/204 ————— 1s 3ms/step - loss: 0.2682 - mae: 0.3572 - val_lo
Epoch 28/100
204/204 ————— 1s 2ms/step - loss: 0.2933 - mae: 0.3710 - val_lo
Epoch 29/100
204/204 ————— 1s 3ms/step - loss: 0.2699 - mae: 0.3586 - val_lo
Epoch 30/100
204/204 ————— 2s 5ms/step - loss: 0.2779 - mae: 0.3647 - val_lo
Epoch 31/100
204/204 ————— 1s 5ms/step - loss: 0.2912 - mae: 0.3697 - val_lo
Epoch 32/100
204/204 ————— 1s 5ms/step - loss: 0.2803 - mae: 0.3612 - val_lo

```



```
Epoch 33/100
204/204 ██████████ 1s 4ms/step - loss: 0.2890 - mae: 0.3663 - val_lo
Epoch 34/100
204/204 ██████████ 1s 3ms/step - loss: 0.2798 - mae: 0.3640 - val_lo
Epoch 35/100
204/204 ██████████ 1s 3ms/step - loss: 0.2720 - mae: 0.3586 - val_lo
Epoch 36/100
204/204 ██████████ 1s 3ms/step - loss: 0.2705 - mae: 0.3563 - val_lo
Epoch 37/100
204/204 ██████████ 1s 3ms/step - loss: 0.2709 - mae: 0.3581 - val_lo
Epoch 38/100
204/204 ██████████ 1s 3ms/step - loss: 0.2722 - mae: 0.3622 - val_lo
Epoch 39/100
204/204 ██████████ 1s 3ms/step - loss: 0.2740 - mae: 0.3597 - val_lo
Epoch 40/100
204/204 ██████████ 1s 3ms/step - loss: 0.2602 - mae: 0.3519 - val_lo
Epoch 41/100
204/204 ██████████ 1s 3ms/step - loss: 0.2779 - mae: 0.3635 - val_lo
Epoch 42/100
204/204 ██████████ 1s 3ms/step - loss: 0.2589 - mae: 0.3505 - val_lo
Epoch 43/100
204/204 ██████████ 1s 2ms/step - loss: 0.2773 - mae: 0.3604 - val_lo
Epoch 44/100
204/204 ██████████ 1s 3ms/step - loss: 0.2586 - mae: 0.3504 - val_lo
Epoch 45/100
204/204 ██████████ 1s 3ms/step - loss: 0.2445 - mae: 0.3390 - val_lo
Epoch 46/100
204/204 ██████████ 1s 3ms/step - loss: 0.2638 - mae: 0.3552 - val_lo
Epoch 47/100
204/204 ██████████ 1s 3ms/step - loss: 0.2590 - mae: 0.3504 - val_lo
Epoch 48/100
204/204 ██████████ 1s 3ms/step - loss: 0.2551 - mae: 0.3447 - val_lo
Epoch 49/100
204/204 ██████████ 1s 3ms/step - loss: 0.2622 - mae: 0.3562 - val_lo
Epoch 50/100
204/204 ██████████ 1s 4ms/step - loss: 0.2620 - mae: 0.3525 - val_lo
Epoch 51/100
204/204 ██████████ 1s 5ms/step - loss: 0.2684 - mae: 0.3572 - val_lo
Epoch 52/100
204/204 ██████████ 1s 4ms/step - loss: 0.2456 - mae: 0.3424 - val_lo
Epoch 53/100
204/204 ██████████ 1s 4ms/step - loss: 0.2538 - mae: 0.3479 - val_lo
Epoch 54/100
204/204 ██████████ 1s 3ms/step - loss: 0.2644 - mae: 0.3532 - val_lo
Epoch 55/100
204/204 ██████████ 1s 3ms/step - loss: 0.2571 - mae: 0.3500 - val_lo
Epoch 56/100
204/204 ██████████ 1s 2ms/step - loss: 0.2421 - mae: 0.3380 - val_lo
Epoch 57/100
204/204 ██████████ 1s 3ms/step - loss: 0.2326 - mae: 0.3311 - val_lo
Epoch 58/100
204/204 ██████████ 1s 2ms/step - loss: 0.2595 - mae: 0.3490 - val_lo
Epoch 59/100
204/204 ██████████ 1s 3ms/step - loss: 0.2573 - mae: 0.3502 - val_lo
Epoch 60/100
204/204 ██████████ 1s 3ms/step - loss: 0.2521 - mae: 0.3452 - val_lo
Epoch 61/100
```

```

Epoch 61/100
204/204 ██████████ 1s 2ms/step - loss: 0.2555 - mae: 0.3469 - val_lo
Epoch 62/100
204/204 ██████████ 1s 3ms/step - loss: 0.2483 - mae: 0.3444 - val_lo
Epoch 63/100
204/204 ██████████ 1s 3ms/step - loss: 0.2616 - mae: 0.3515 - val_lo
Epoch 64/100
204/204 ██████████ 1s 2ms/step - loss: 0.2606 - mae: 0.3466 - val_lo
Epoch 65/100
204/204 ██████████ 1s 3ms/step - loss: 0.2354 - mae: 0.3334 - val_lo
Epoch 66/100
204/204 ██████████ 1s 3ms/step - loss: 0.2412 - mae: 0.3438 - val_lo
Epoch 67/100
204/204 ██████████ 1s 2ms/step - loss: 0.4903 - mae: 0.4642 - val_lo
Epoch 68/100
204/204 ██████████ 1s 3ms/step - loss: 0.3209 - mae: 0.4001 - val_lo
Epoch 69/100
204/204 ██████████ 1s 2ms/step - loss: 0.3047 - mae: 0.3791 - val_lo
Epoch 70/100
204/204 ██████████ 1s 3ms/step - loss: 0.3114 - mae: 0.3892 - val_lo
Epoch 71/100
204/204 ██████████ 1s 3ms/step - loss: 0.2741 - mae: 0.3585 - val_lo
Epoch 72/100
204/204 ██████████ 1s 4ms/step - loss: 0.2868 - mae: 0.3686 - val_lo
Epoch 73/100
204/204 ██████████ 1s 4ms/step - loss: 0.2568 - mae: 0.3500 - val_lo
Epoch 74/100
204/204 ██████████ 1s 5ms/step - loss: 0.2729 - mae: 0.3535 - val_lo
Epoch 75/100
204/204 ██████████ 1s 3ms/step - loss: 0.2663 - mae: 0.3556 - val_lo
Epoch 76/100
204/204 ██████████ 1s 2ms/step - loss: 0.2711 - mae: 0.3541 - val_lo
Epoch 77/100
204/204 ██████████ 1s 2ms/step - loss: 0.2605 - mae: 0.3528 - val_lo
Epoch 78/100
204/204 ██████████ 1s 2ms/step - loss: 0.2611 - mae: 0.3511 - val_lo
Epoch 79/100
204/204 ██████████ 1s 2ms/step - loss: 0.2468 - mae: 0.3432 - val_lo
Epoch 80/100
204/204 ██████████ 1s 2ms/step - loss: 0.2612 - mae: 0.3501 - val_lo
Epoch 81/100
204/204 ██████████ 1s 3ms/step - loss: 0.2420 - mae: 0.3373 - val_lo
Epoch 82/100
204/204 ██████████ 1s 3ms/step - loss: 0.2516 - mae: 0.3434 - val_lo
Epoch 83/100
204/204 ██████████ 1s 3ms/step - loss: 0.2455 - mae: 0.3370 - val_lo
Epoch 84/100
204/204 ██████████ 1s 3ms/step - loss: 0.2464 - mae: 0.3431 - val_lo
Epoch 85/100
204/204 ██████████ 1s 3ms/step - loss: 0.2621 - mae: 0.3547 - val_lo
Epoch 86/100
204/204 ██████████ 1s 2ms/step - loss: 0.2451 - mae: 0.3437 - val_lo
Epoch 87/100
204/204 ██████████ 1s 2ms/step - loss: 0.2440 - mae: 0.3435 - val_lo
Epoch 88/100
204/204 ██████████ 1s 2ms/step - loss: 0.2459 - mae: 0.3420 - val_lo
Epoch 89/100

```

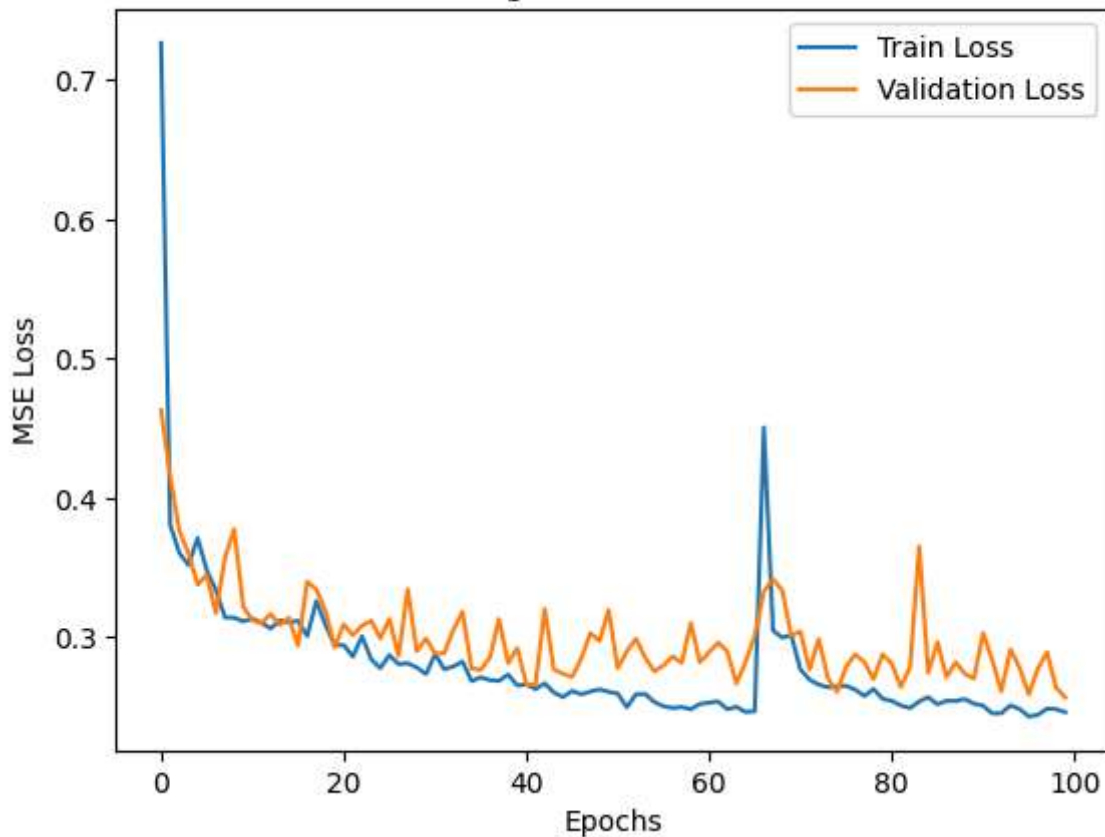
```
204/204 1s 3ms/step - loss: 0.2674 - mae: 0.3523 - val_lo
Epoch 90/100
204/204 1s 2ms/step - loss: 0.2581 - mae: 0.3464 - val_lo
Epoch 91/100
204/204 1s 3ms/step - loss: 0.2468 - mae: 0.3414 - val_lo
Epoch 92/100
204/204 1s 2ms/step - loss: 0.2426 - mae: 0.3414 - val_lo
Epoch 93/100
204/204 1s 4ms/step - loss: 0.2548 - mae: 0.3463 - val_lo
Epoch 94/100
204/204 1s 4ms/step - loss: 0.2405 - mae: 0.3394 - val_lo
Epoch 95/100
204/204 1s 5ms/step - loss: 0.2668 - mae: 0.3506 - val_lo
Epoch 96/100
204/204 1s 4ms/step - loss: 0.2418 - mae: 0.3357 - val_lo
Epoch 97/100
204/204 1s 4ms/step - loss: 0.2385 - mae: 0.3379 - val_lo
Epoch 98/100
204/204 1s 3ms/step - loss: 0.2437 - mae: 0.3393 - val_lo
Epoch 99/100
204/204 1s 3ms/step - loss: 0.2528 - mae: 0.3432 - val_lo
Epoch 100/100
204/204 1s 3ms/step - loss: 0.2483 - mae: 0.3403 - val_lo
194/194 0s 2ms/step - loss: 0.2983 - mae: 0.3577
194/194 0s 1ms/step
```

Test MAE: 0.3554988205432892

RMSE: 0.5351141619336767

R2 Score: 0.781837397122352

Training vs Validation Loss



Actual vs Predicted Housing Prices

```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)

print("\nDataset Columns:")
print(df.columns.tolist())

print("\nFirst 5 Rows:")
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42
)

# =====
# 5. Feature Scaling
# =====
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("\nStandardScaler Parameters:")
```